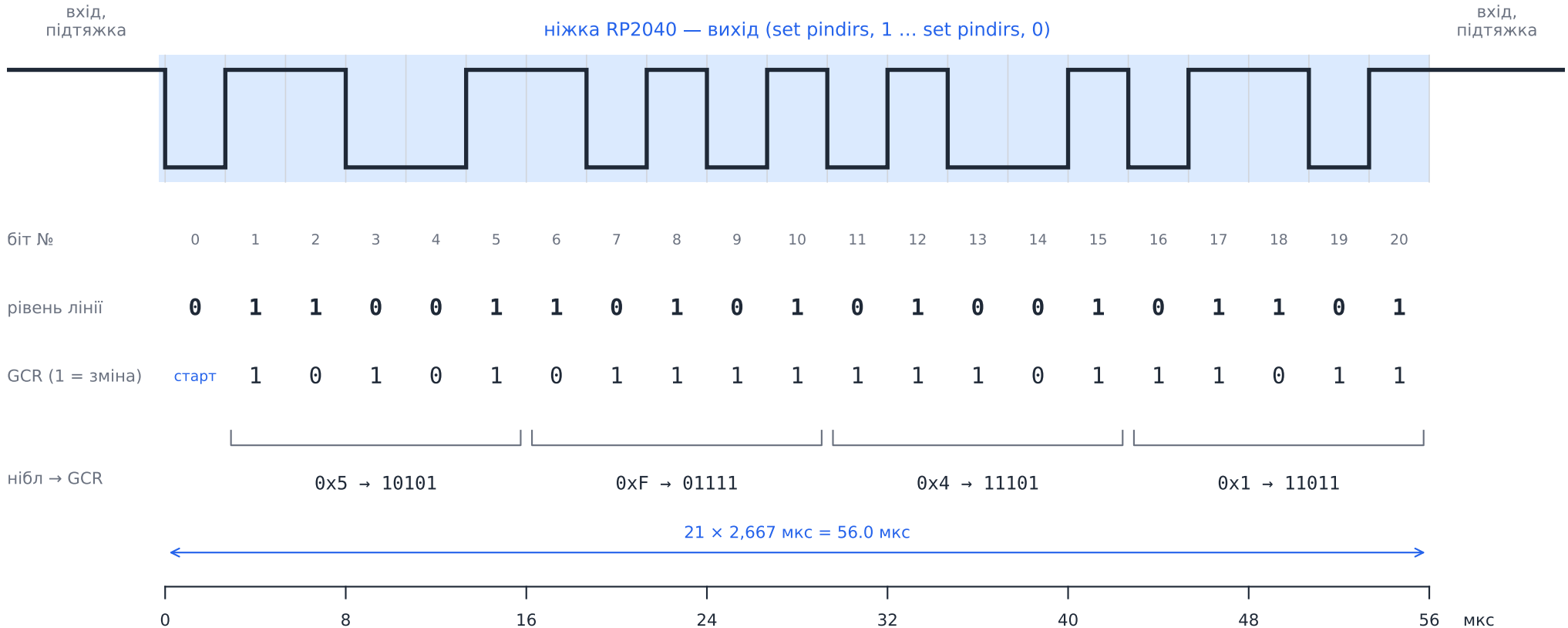


Відповідь bidirectional DShot300 для eRPM 0x5F4 (30 000 eRPM)

e12 = 0x5F4 → CRC = 0x1 → слово 0x5F41 → GCR → NRZI + старт-біт → 21 біт, у TX FIFO: 0x66A96800 · біт = 32 такти = 2,667 мкс



- Лінія в спокої висока. Біт 0 — старт-біт: перший спад, за яким FC знаходить початок відповіді.
- NRZI: GCR-біт 1 — рівень змінюється на межі біта, 0 — лишається. Рівень тримається від одного out pins до наступного.
- Пауза від кінця кадру FC до першого спаду ≈ 29-31 мкс (залежить від останнього біта кадру).
- CPU кладе в TX FIFO вже готові рівні: 0x66A96800 (21 біт у бітах 31...11, зсув OSR вліво).
- Після біта 20 PIO виконує set pins, 1 → set pindirs, 0: лінія активно піднімається, потім її тримає підтяжка.

Як кодується відповідь **bidirectional DShot**

Покроковий розбір на прикладі 30 000 eRPM: від обертів мотора до 32-бітного слова 0x66A96800, яке CPU кладе в TX FIFO. На кожному кроці — що робимо, навіщо і що виходить.

Загальна схема

eRPM → період, мкс → e12 (12 біт)	крок 1–2
e12 → + CRC (4 біти) = 16-бітне слово	крок 3
16 біт → 4 нібли → GCR → 20 біт	крок 4
20 біт → NRZI + старт-біт → 21 рівень	крок 5
21 рівень → << 11 → слово для TX FIFO	крок 6

Кроки 1–3 стосуються змісту відповіді: що саме ми повідомляємо FC. Кроки 4–6 — того, як це передати одним проводом так, щоб FC надійно прийняв. Останній розділ проходить той самий шлях у зворотному напрямку, як це робить FC, — це найкращий спосіб перевірити себе.

Крок 1. Від обертів до періоду

ESC не рахує оберти напряму. Він бачить комутації: моменти, коли перемикає фази мотора. Найпростіше, що він може виміряти, — час між ними, тобто період одного електричного оберту. Тому DShot-телеметрія передає саме період у мікросекундах, а оберти FC обчислює сам.

Електричні оберти (eRPM) — це механічні оберти, помножені на кількість пар полюсів мотора. Для мотора з 14 магнітами (7 пар полюсів) $30\,000\text{ eRPM} \approx 4\,286$ механічних об/хв. FC знає кількість полюсів з налаштування `motor_poles` і перераховує сам; ESC про це не думає.

Період у мікросекундах: хвилина = 60 000 000 мкс, ділимо на кількість обертів за хвилину:

$$\text{період} = 60\,000\,000 / \text{eRPM} = 60\,000\,000 / 30\,000 = 2000\text{ мкс}$$

Що швидше крутиться мотор, то менший період. Це зручно: на великих обертах, де потрібна точність, число маленьке і вміщується без втрат (див. крок 2).

Крок 2. Упаковка періоду в 12 біт (e12)

На все значення є лише 12 біт, а період може бути і 100 мкс, і 60 000 мкс. Тому його записують як число з плаваючою комою в мініатюрі: 3 біти експоненти *e* і 9 біт мантиси *m*.

біт:	11	10	9	8	7	6	5	4	3	2	1	0
	e	e	e	m	m	m	m	m	m	m	m	m

період = $m \ll e$ (тобто $m \cdot 2^e$)

Алгоритм кодування: поки m не влізєть у 9 біт ($m > 511$), ділимо його навпіл (зсув вправо) і збільшуємо e на 1. Для нашого прикладу:

e	m	m у 9 біт?
0	2000	ні, > 511
1	1000	ні, > 511
2	500	так

```
e12 = (e << 9) | m = (2 << 9) | 500
      = 1024 + 500 = 1524 = 0x5F4
      = 010 111110100 (eee mmmmmmmmm)
```

Ціна такої упаковки — округлення: кожен зсув вправо відкидає молодший біт. Чим більший період (повільніший мотор), тим грубіший крок. Кілька прикладів:

eRPM	період	e	m	e12	m << e	eRPM у FC
100 000	600	1	300	0x32C	600	100 000
45 000	1333	2	333	0x54D	1332	45 045
30 000	2000	2	500	0x5F4	2000	30 000
10 000	6000	4	375	0x977	6000	10 000
3 000	20000	6	312	0xD38	19968	3 004
1 000	60000	7	468	0xFD4	59904	1 001

- Значення 0xFFFF ($e = 7$, $m = 511$) домовлено вважати «мотор стоїть». Його ж віддаємо, коли $period = 0$.
- Тому найповільніше значення, яке FC ще сприймає як оберти, — 0xFFE ($510 << 7 = 65\,280$ мкс ≈ 919 eRPM). Усе повільніше (≤ 917 eRPM) кодується як 0xFFFF. Докладніше — у `erpm-lower-limit.md`.
- Похибка ніколи не перевищує $1/256 \approx 0,4\%$: після нормалізації $m \geq 256$, коли $e > 0$.

Крок 3. Контрольна сума (CRC)

Відповідь іде проводом поруч із силовими лініями мотора, тож біти можуть спотворитися. Щоб FC міг відкинути зіпсовану відповідь, до 12 біт додається 4-бітна контрольна сума.

Рахується вона дуже просто: 12 біт ділимо на три нібли (по 4 біти) і складаємо їх через XOR. У коді це записують як $v \wedge (v >> 4) \wedge (v >> 8)$ і беруть молодші 4 біти:

```
нібли 0x5F4:  5      F      4
                0101  1111  0100

  0100  (0x4)
^ 1111  (0xF)
^ 0101  (0x5)
= 1110  (0xE)
інверсія: ~1110 = 0001 → CRC = 0x1
```

Остання дія — інверсія (NOT). Вона є тільки в двонапрямному режимі: як і рівні лінії, CRC тут «перевернута» відносно звичайного DShot. Так само інвертована і CRC у кадрах, які шле FC (див. `frame_ok()` у плані). Забудете інверсію — FC відкидатиме кожну відповідь як бити.

CRC дописується в молодші 4 біти. Виходить 16-бітне слово:

```
слово = (e12 << 4) | CRC = (0x5F4 << 4) | 0x1 = 0x5F41
        = 010 111110100 0001
          eee mmmmmmmmm cccc
```

Крок 4. GCR: 4 біти → 5 біт

Тепер 16 біт треба передати. Проблема в тому, що FC не має окремого тактового сигналу: він бачить лише фронти (зміни рівня) і за ними підлаштовує свій відлік бітів. На кроці 5 ми закодуємо біти так, що «1» = фронт, а «0» = відсутність фронту. Якщо в даних трапиться довга серія нулів, на лінії довго не буде жодного фронту, і годинник FC, який трохи відрізняється від нашого, «попливе» — він може порахувати, наприклад, шість нулів замість п'яти.

GCR (Group Coded Recording) розв'язує це так: кожен нібл (4 біти, 16 варіантів) замінюється 5-бітним кодом із таблиці. З 32 можливих 5-бітних кодів вибрано 16 таких, що в будь-якій послідовності кодів підряд ніколи не трапляється більше двох нулів. Отже, фронт на лінії з'являється щонайменше кожні 3 біти.

нібл	GCR	hex	нібл	GCR	hex
0x0	11001	0x19	0x8	11010	0x1A
0x1	11011	0x1B	0x9	01001	0x09
0x2	10010	0x12	0xA	01010	0x0A
0x3	10011	0x13	0xB	01011	0x0B
0x4	11101	0x1D	0xC	11110	0x1E
0x5	10101	0x15	0xD	01101	0x0D
0x6	10110	0x16	0xE	01110	0x0E
0x7	10111	0x17	0xF	01111	0x0F

Чому не буває трьох нулів підряд: жоден код не має двох нулів на початку чи в кінці, а всередині коду максимум два нулі підряд (11001, 10010). На стику двох кодів — щонайбільше один нуль у кінці першого плюс один на початку другого.

Застосовуємо таблицю до кожного нібла слова, починаючи зі старшого:

нібл	значення	GCR
перший (біти 15-12)	0x5 = 0101	10101
другий (біти 11-8)	0xF = 1111	01111
третій (біти 7-4)	0x4 = 0100	11101
четвертий (біти 3-0)	0x1 = 0001	11011

```
GCR-потік (20 біт): 10101 01111 11101 11011
```

Ціна — 25 % накладних витрат: 16 біт даних займають 20 біт на лінії. Саме тому швидкість відповіді (375 кбіт/с) вища за швидкість команд DShot300 у 5/4 рази.

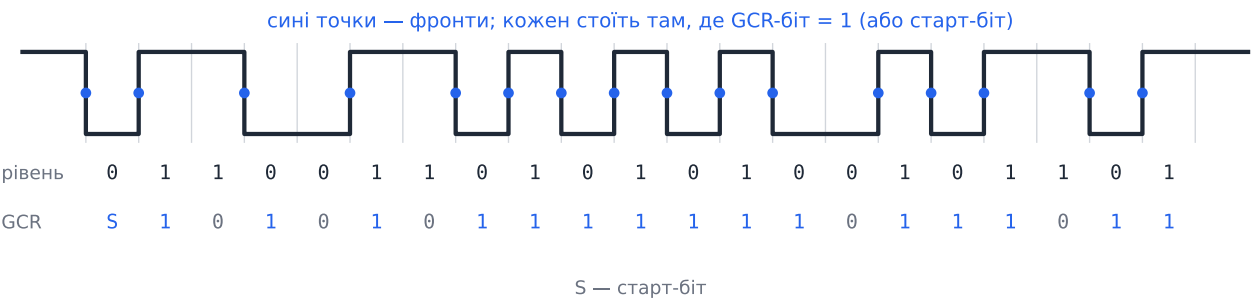
Крок 5. NRZI і старт-біт: від бітів до рівнів лінії

NRZI (Non-Return-to-Zero Inverted) — це правило, за яким біти перетворюються на рівні: біт 1 — змінити рівень на протилежний, біт 0 — залишити рівень як є. Інформація, отже, закодована не в рівнях, а в моментах фронтів. Тому FC байдуже, з якого рівня ми

почали, — важливо лише, де відбуваються зміни.

Перед даними йде старт-біт: лінія в спокої висока, і ми опускаємо її в 0. Цей перший спадаючий фронт FC використовує як точку відліку — від нього він починає відміряти біти. Далі кожен із 20 GCR-бітів по черзі застосовуємо до поточного рівня:

рівень №	GCR-біт	був рівень	став	дія
старт	—	1 (спокій)	0	спад: початок відповіді
1	1	0	1	змінити
2	0	1	1	залишити
3	1	1	0	змінити
4	0	0	0	залишити
5	1	0	1	змінити
6	0	1	1	залишити
7	1	1	0	змінити
8	1	0	1	змінити
9	1	1	0	змінити
10	1	0	1	змінити
11	1	1	0	змінити
12	1	0	1	змінити
13	1	1	0	змінити
14	0	0	0	залишити
15	1	0	1	змінити
16	1	1	0	змінити
17	1	0	1	змінити
18	0	1	1	залишити
19	1	1	0	змінити
20	1	0	1	змінити



21 рівень лінії: 0 11001101010100101101

Перевірка з боку синхронізації: інтервали між фронтами (у бітах) — 1, 2, 2, 2, 1, 1, 1, 1, 1, 1, 2, 1, 1, 2, 1. Найдовший — 2 біти, як і обіцяв GCR. Після останнього рівня PIO виконує set pins, 1, тож лінія повертається у спокій.

Чому кодуємо NRZI на CPU, а не в PIO: PIO-програма тоді лише «викладає» готові рівні по одному за 32 такти, без розгалужень. Таймінг кожного біта гарантовано однаковий, а програма коротка.

Крок 6. Вирівнювання в слово для TX FIFO

PIO забирає з TX FIFO одне 32-бітне слово командою pull noblock у регістр OSR, а потім 21 раз виконує out pins, 1. OSR налаштований на зсув уліво, тому кожен out бере найстарший біт, що залишився. Отже, перший рівень (старт-біт) має бути в біті 31, другий — у біті 30, і так далі до біта 11. Молодші 11 біт PIO не прочитає — там можуть бути нулі.

```
21 рівень:      011001101010100101101
<< 11:         01100110101010010110100000000000
по 4 біти:     0110 0110 1010 1001 0110 1000 0000 0000
hex:           6    6    A    9    6    8    0    0
слово:         0x66A96800
```

Якщо CPU не встиг покласти слово, pull noblock бере вміст регістра X, а там після циклу прийому 0xFFFFFFFF. Це 21 одиниця: лінія лишається високою, старт-біта немає, і FC просто зараховує пропущену телеметрію. Нічого не ламається.

Перевірка: декодуємо назад, як FC

FC отримує 21 рівень і проходить усі кроки у зворотному порядку. Якщо вручну отримати вихідні 30 000 eRPM, кодування правильне.

- Сусідні рівні через XOR: де рівень змінився — 1, де ні — 0. Виходить GCR-потік 10101 01111 11101 11011.
- Кожні 5 біт шукаємо у таблиці GCR: 10101 → 0x5, 01111 → 0xF, 11101 → 0x4, 11011 → 0x1. Слово 0x5F41. (Код, якого немає в таблиці, — одразу помилка.)
- CRC: XOR ніблів 0x5F4, інверсія = 0x1 — збігається.
- $e12 = 0x5F4$: $e = 2$, $m = 500$, період = $500 \ll 2 = 2000$ мкс.
- $eRPM = 60\,000\,000 / 2000 = 30\,000$.

Те саме в коді (Rust)

Функції повторюють кроки 2–6. Їх зручно тримати в окремому `no_std`-модулі й тестувати звичайним `cargo test` на комп'ютері (крок 5 плану).

```
const GCR: [u8; 16] = [
    0x19, 0x1B, 0x12, 0x13, 0x1D, 0x15, 0x16, 0x17,
    0x1A, 0x09, 0x0A, 0x0B, 0x1E, 0x0D, 0x0E, 0x0F,
];

/// Крок 2: період (мкс) → e12.
fn encode_period(period_us: u32) -> u16 {
    if period_us == 0 {
        return 0xFFFF;
    }
    let mut m = period_us;
    let mut e = 0;
    while m > 0x1FF {
        m >>= 1;
        e += 1;
    }
    if e > 7 {
        return 0xFFFF; // занадто повільно — мотор стоїть
    }
    ((e << 9) | m) as u16
}

/// Кроки 3–6: e12 → слово для TX FIFO.
fn reply_word(e12: u16) -> u32 {
    let v = e12 & 0xFFFF;
    let crc = !(v ^ (v >> 4) ^ (v >> 8)) & 0x0F; // крок 3
    let word = (v << 4) | crc;

    let mut gcr: u32 = 0; // крок 4
    for i in (0..4).rev() {
        let nib = (word >> (i * 4)) & 0x0F;
        gcr = (gcr << 5) | GCR[nib as usize] as u32;
    }

    let mut level = 0u32; // старт-біт // крок 5
    let mut out = level;
    for i in (0..20).rev() {
        level ^= (gcr >> i) & 1; // 1 = зміна рівня
        out = (out << 1) | level;
    }
    out << 11 // 21 рівень у бітах 31...11 // крок 6
}

#[test]
fn reply_matches_waveform() {
    assert_eq!(encode_period(2000), 0x5F4);
    assert_eq!(reply_word(0x5F4), 0x66A9_6800);
}
```